CSCI218: Foundations of Artificial Intelligence

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Lab 4 Assessed Lab NLP

Agenda

Part A (1.5 hr) Moodle Assessed Lab 1

- https://colab.research.google.com/drive/1qwmPH6Hwfvkoglv Qp9nqTrXg9anv3LKa?usp=sharing

Your tasks:

1. Run the code
2. Complete answering the questions listed at the first cell of the Colab Lab code.
3. Export the Colab code into **PDF** and **Submit** to Moodle within your respective Lab session.
4. I will receive a notification within your respective Lab session.

Questions to be answered in Lab 4

This lab enables you to answer the following questions:

1. Describe the key steps for data preparation and feature extraction.
2. Report the overall classification results, including precision, recall, and f1-score. Explain the meaning of these criteria.
3. Plot the confusion matrix for your classification result. Find the pair of classes that confuses the classifier most. Is this result consistent with your expectation?
4. Based on the confusion matrix, report the individual accuracy scores for each class.
5. Train a Complement Naive Bayes classifier and compare its classification results with those of Multinomial Naive Bayes.
6. What did GenAI help you with in the above tasks, and what did you complete independently?
7. Which part required human judgement or personal experience that GenAI could not replace?

Lab 4 Assessed Lab NLP

Part B (1 hr) : Core classical NLP concepts:

- BOW
- N-gram
- POS tagging
- Generative Models (e.g. Naive Bayes) vs Discriminative Models (e.g., Logistic Regression)
- Word Embedding
- RNN / LSTM
- Sequence to Sequence model e.g. Machine Translation
- Attention vs Self-attention
- Masked Language Models

NLP pipelines

	Traditional NLP	Modern NLP (Deep Learning / Transformer Era)
Input sentences	Raw text sentences	Raw text sentences
Text Processing	Cleaning / Normalisation (lowercase, punctuation removal, optional stopword removal)	Minimal normalisation (often keeps punctuation, stopwords, casing)
Tokenisation	Word-level tokenisation	Subword tokenisation (e.g., WordPiece, BPE)
Feature Representation	Sparse vectors (BoW, N-grams, TF-IDF)	Dense embeddings (contextual vector representations)
Classifier Type	Classical ML models (Naive Bayes, Logistic Regression, SVM)	Neural models (LSTM, Transformer, etc.)

BOW

E.g., Input sentence: "stocks rallied last Monday and also the Monday before".

Build a dictionary from the training corpus e.g., ["stocks", "rallied", "monday", "flood", "rain"]

Two classes:

1. Business

2. Weather

Tokenize the sentence into a vector :

"0" means the word is not in dictionary;

"1" means the word is in dictionary;

Convert words into feature counts

Ignore word count: Input vector = [1,1,1,0,0]

Take word count into consideration: Input vector = [1, 1, 2, 0, 0]

Naïve Bayes Classifier

- The BOW vector becomes the feature inputs to the Naïve Bayes classifier, where each value represents the frequency of a word used to compute class probabilities.
- E.g., **feature input = [1,1,2,0,0]**
- Naïve Bayes uses this vector to compute the score per class.
- Prediction rule: Choose class with higher score.
  - Score(Business)
    - = P(Business)
    - × P(stocks | Business)^1
    - × P(rallied | Business)^1
    - × P(monday | Business)^2
  - Score(Weather)
    - = P(Weather)
    - × P(stocks | Weather)^1
    - × P(rallied | Weather)^1
    - × P(monday | Weather)^2

Dictionary from training corpus
["stocks", "rallied", "monday", ”flood", ”rain"]

Two classes:
1. Business
2. Weather

E.g., Input sentence:
“stocks rallied last Monday and also the Monday before”.

N-grams

- When classifier ignores word order: (these sentences carry different meaning)
  1. Stocks rallied last Monday.
  2. Monday rallied stocks.
- An N-gram model assumes that a word depends only on the previous N-1 words, instead of the entire sentence history.
- When N=1, it is unigram.
- BOW = unigram.

N-grams

Input: “stocks rallied last Monday”

- N=1 (Unigram)
  - (stocks, rallied, last, monday)
- N=2 (Bigram)
  - (stocks, rallied)
  - (rallied, last)
  - (last, monday)
- N=3 (Trigram)
  - (stocks, rallied, last)
  - (rallied, last, monday)

N-grams

The pipeline is **identical** to BoW:

1. Extract N-grams from sentence
2. Convert them into feature counts
3. Compute class scores

But now the likelihood is: $P(c) \prod P(\text{n-gram}_i \mid c)$

Input: "stocks rallied last Monday"

E.g., Bigram: Instead of P("stocks" | Business), you will now use P("stocks, rallied" | Business)

For Business:

$$P(\text{Business}) \times P(\text{stocks} \mid \text{Business}) \times P(\text{rallied} \mid \text{Business}) \times P(\text{monday} \mid \text{Business})$$

Modify accordingly

For Weather:

$$P(\text{Weather}) \times P(\text{stocks} \mid \text{Weather}) \times P(\text{rallied} \mid \text{Weather}) \times P(\text{monday} \mid \text{Weather})$$

N-grams

- Why not increase N forever?

N	Problem
1	No order (*N=1* => *BOW*)
2	Manageable
3	Sparse
4+	Explosion of features

BOW (N=1) vs N-grams

	Bag-of-Words	**N-grams**
Word order	Ignored	Local
Feature size	Small	Large
Data needed	Low	Higher
Long-range dependency	Ignored	Ignored
Typical use	Baseline text classification	Early language modelling

Generative Model vs Discriminative Model in NLP

Generative model e.g., Naïve Bayes

- Corpus: “market, profit, today, goal, team”
- During training, it learns:

Class	P(Class)
Business	0.4
Sports	0.3
Weather	0.3

Word	P(word | Business)
market	0.08
profit	0.06
today	0.02

Word	P(word | Sports)
goal	0.09
team	0.07
today	0.04

Generative vs Discriminative Model in NLP

- During testing:
  - For input: “today market rises”
  - Compute: P(Business) × P(today|Business) × P(market|Business) × …
  - Do this for all classes.
  - Select the class with the highest score.
- Why NB is called Generative?
  - NB stores the probabilities of word per class during learning.
  - If you sample words using P(word | Business), you can generate a business like sentence.
  - That is the reason it is called a Generative Model.

Generative vs Discriminative Model in NLP

- Discriminative model e.g., Logistic Regression
- Corpus: “market, profit, today, goal, team”
- During training, model learns:

Word	Weight
today	+0.5
market	+2.0
profit	1.8
goal	-2.2
rain	-1.5

w = [0.5, 2.0, 1.8, -2.2, -1.5]
b = -0.3

Generative vs Discriminative Model in NLP

- During testing:
  - Input sentence = “today market profit”; x = [1,1,1,0,0]
  - z = w·x + b
  - = [0.5, 2.0, 1.8, -2.2, -1.5].[1,1,1,0,0] + [-0.3]
  - = 4
- Logistic Regression p(class) = sigmoid(z) or softmax(z)
- Assume binary class: (Business or Not Business)

- If z = 4, then p(business | x) = 0.982
- That means 98.2% confidence that input senten

$$P(Business|X) = \frac{1}{1+e^{-z}}$$

- Logistic regression does not store the word probabilities per class. It uses weights and bias to create a decision boundary for classification. That’s why it is discriminative.

Generative vs Discriminative Model in NLP

	Naive Bayes	**Logistic Regression**
Model type	Generative	Discriminative
Learns	P(X|class)	P(class|X)
Learns language model?	Yes	No
Stores	Word probabilities per class	Feature weights
Data requirement	Low (only count frequency to generate probabilities.)	Medium–High (needs data to learn w, b in order to derive decision boundary)
Handles small dataset	Very good	Average
Decision boundary	Indirect	Direct
Can generate text	Theoretically yes	No
Correlation between words	Assume independent e.g. p(w1 and w2 | class) = p(w1|class) * p(w2 | class)	Learns correlation during training via weights.

Part-of-speech (POS)

- Part-of-Speech (POS) tagging assigns grammatical categories (e.g., noun, verb, adjective) to each word based on its context, adding syntactic structure to raw text.
- POS tagging = "Add grammar labels to words"

POS tagging example

- Input sentence: “stocks rallied on Monday and also last Monday”
- Step 0: Text processing (e.g., cleaning & normalization)
- Step 1: Tokenize
- Step 2: Assign POS tags (using context)

Word	POS tag	Tagged sequence
Stocks	Noun	stocks/NOUN
Rallied	Verb	rallied/VERB
on	Preposition	on/PREP
Monday	PropN / Noun	Monday/PROPN
and	Conjunction	and/CONJ
also	Adverb	also/ADV
last	Adjectives	last/ADJ
Monday	PropN / Noun	Monday/PROPN

Part-of-speech (POS)

Input text

→ Text processing

→ Tokenisation

→ Feature extraction (BoW / N-grams)

→ **POS tagging**

→ Higher-level structure prediction

- Chunking
- Named Entity Recognition
- Dependency parsing

→ Downstream task / application

- Classification
- Question Answering
- Machine Translation

Higher level structure prediction

Task	Input	Output	Example
POS tagging	words + context	POS tag per word	**Input:** stocks rallied on Monday **Output:** stocks/NOUN rallied/VERB on/PREP Monday/PROPN
Chunking (phrase detection)	words + POS tags	phrase label per word	**Input:** last/ADJ Monday/PROPN **Output:** last B-NP, Monday I-NP (noun phrase)
Named Entity Recognition (NER)	words + POS tags	entity label per word/span	**Input:** Monday/PROPN **Output:** Monday → DATE
Dependency parsing	words + POS tags	dependency relations (tree)	**Input:** stocks/NOUN rallied/VERB on/PREP Monday/PROPN **Output:** rallied → subject → stocks rallied → time-modifier → Monday

Is POS tagging used in LLM?

- POS tagging is still used today, but in modern neural NLP systems, grammatical structure is often learned implicitly rather than produced as a separate pipeline stage.
- Examples:
  - Traditional POS tagging: Explicit POS tagging → Next task
  - Modern POS tagging: Neural model learns POS-like structure automatically